LEARNER COMPANION · READ ALONGSIDE THE SLIDES

Python & FastAPI, explained properly

Every concept from the three-hour session, written out in full — what it is, why it exists, and the literal questions people are too polite to ask out loud. Nothing here assumes you already know the answer.

HOW TO USE THIS DOCUMENT

Read a section before its segment if you want to feel ahead, or after it if you want things to settle. The grey boxes marked **No stupid question** answer the things that genuinely confuse people the first time — including several that are never covered on a slide. You do not need to memorise any of it; you need to be able to find it again.

PART 0

The mental model

What "the bridge" means

Module 1 taught the back of the house: Python, APIs, data. Module 2 taught the front: the web page a person actually looks at. Those two halves are usually taught separately and then, at some point, you are expected to connect them. This session is that connection, done once, deliberately, at the smallest possible scale.

Everything you build today is a loop: a person types something into a page, the page sends it to a Python service, the service stores it in a database, and the page reads it back. That loop is the shape of almost every product you will ever work on. Week 1 makes the loop bigger. It does not make it different.

The running example

We build a loan-application intake service. Someone submits an application; staff read the list back. The whole API is two endpoints — `POST /applications` to submit one, `GET /applications` to list them. It is deliberately trivial: you should never be stuck thinking about loans, only about the tools.

NO STUPID QUESTION

What is an API, in plain words?

A way for one program to ask another program for something, over a network, in a format both agree on. Your web page cannot reach into the database itself; it asks the API, and the API does the reaching. The "agreement" — which addresses exist, what they accept, what they return — is the whole job. **REST** is just a common set of conventions for that agreement: use URLs to name things (`/applications`) and HTTP verbs to say what you are doing to them (`GET` to read, `POST` to create).

PART 1 · SEGMENT 01

Your machine and the four tools

The session opens by running four commands. This is not ceremony — it is the cheapest possible way to find out that something is broken. An install problem discovered at minute five is a footnote; the same problem discovered during the checkpoint costs you the session.

```
python --version        # expect 3.11 or later
docker --version
docker compose version
git --version
```

Docker, and why Compose is a separate command

This is the single most common point of confusion in the whole session, so it is worth being precise.

Docker is the engine that runs containers. A container is a packaged, isolated copy of a program together with everything it needs to run — its operating-system libraries, its language runtime, its dependencies. You give Docker an *image* (the recipe, e.g. `postgres:16-alpine`) and it gives you a running *container* (the live thing). The Docker CLI, on its own, deals with **one** container at a time: build this image, run that container, stop it.

Docker Compose is a layer on top for running **several** containers as one system. You describe the services in a `docker-compose.yml` file — our three are `db`, `api` and `web` — and Compose starts them all with one command, puts them on a shared private network so they can find each other by name, wires up their volumes and environment variables, and enforces start-up order. Doing that by hand with plain `docker run` commands is possible, and miserable.

NO STUPID QUESTION

So do I install Docker Compose separately or not?

On Docker Desktop — what we use — **no**. Compose ships with it. The reason people believe otherwise is history: the original Compose (v1) really was a separate program you installed yourself, and it was called `docker-compose`, with a hyphen. The current Compose (v2) is a plugin built into the Docker CLI, and it is called `docker compose`, with a **space**. Same tool, new packaging.

So why do we check `docker compose version` as its own command? Because a working `docker` does not, by itself, prove the Compose plugin is present and callable. That is a separate thing that can be missing — most often on Linux, where you install the engine on its own and may need to add the Compose plugin package. One extra command now beats discovering it at 1:45.

If you find older tutorials typing `docker-compose up`, translate to `docker compose up` and carry on.

NO STUPID QUESTION

How is a container different from a virtual machine?

A virtual machine simulates a whole computer, including its own operating-system kernel — heavy, slow to start, gigabytes. A container shares your machine's kernel and isolates only the program and its files — light, starts in a second, megabytes. That difference is why running three containers on a laptop during a class is reasonable, and running three VMs would not be.

NO STUPID QUESTION

Image or container — which is which?

An image is the frozen recipe on disk; a container is a running instance of it. One image can produce many containers, the way one class produces many objects. `docker pull` fetches an image; `docker run` starts a container from it. When we "pre-pull `postgres:16-alpine` during the break", we are downloading the recipe early so that starting it later is instant.

Python, pip and Git

Python 3.11 or later matters for real reasons, not fashion. It supports the modern type syntax we use (`list[Applicant]` and `float | None` without extra imports), it is meaningfully faster than 3.8-era versions, and current FastAPI and Pydantic versions expect

it. **pip** is Python's package installer — it fetches libraries from the public index and puts them where your Python can import them. **Git** is version control: it records snapshots of your project over time and lets you push them to a shared server so the work exists somewhere other than your laptop.

NO STUPID QUESTION

Is it `python` or `python3`? Mine says 2.7!

On some macOS and Linux setups, `python` points at an ancient Python 2 (or nothing at all) and the one you want is `python3`. Use whichever reports 3.11+. Once you are inside an activated virtual environment this stops mattering — there, `python` reliably means *your project's* Python.

PART 2 · SEGMENT 02

Virtual environments

The problem they solve

Imagine two projects on one laptop. Project A needs version 1 of a library; project B needs version 2. If both install into the same shared Python, installing for B silently breaks A — and you find out days later, from an error that makes no sense. A virtual environment gives each project *its own* Python and *its own* packages, in a folder inside the project. Nothing is shared, so nothing can be broken from outside.

```
python -m venv .venv               # create the environment
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install fastapi "uvicorn[standard]" pydantic pytest httpx
pip freeze > requirements.txt      # capture exact versions
```

What creation and activation actually do

Creating makes a real folder called `.venv/` containing a Python interpreter and an empty package directory. **Activating** does something much smaller than people assume: it edits a couple of environment variables in your *current terminal session* so that the words `python` and `pip` now resolve to the ones inside `.venv/` instead of the machine's. That is all. It installs nothing, changes no files, and touches nothing outside that one shell.

This is why the prompt changes to show `(.venv)` — it is a reminder of which Python you are currently talking to.

NO STUPID QUESTION

I restarted my machine. Is my virtual environment gone?

No. The environment is a folder of files on your disk, and files survive restarts like any other folder. Your installed packages are all still there.

What *is* lost is the **activation**, because activation only ever lived in that one terminal session's environment variables. Close the terminal, open a new tab, or restart the machine, and you simply re-run:

```
cd loan-intake
source .venv/bin/activate
```

Nothing to reinstall, nothing to rebuild — one command. If you skip it, your imports will fail with `ModuleNotFoundError`, because you are talking to the machine's Python, which never saw your `pip install`. That error is almost always this, and nothing more sinister.

NO STUPID QUESTION

Can I just delete `.venv/`? And why the leading dot?

Yes — deleting the folder is a complete, clean uninstall, with no leftovers anywhere on your system. You can rebuild it in seconds with `python -m venv .venv` and `pip install -r requirements.txt`. That disposability is the point: the environment is derived, never precious. The leading dot just marks it hidden on macOS and Linux so it stays out of your way in file listings.

NO STUPID QUESTION

What does the `-m` in `python -m venv` mean?

"Run the installed module named `venv` as a program." It guarantees you are using the `venv` that belongs to *that specific* Python, which matters when several Pythons exist on one machine. The same trick works elsewhere — `python -m pytest` runs the pytest belonging to the Python you invoked.

requirements.txt — how the environment travels

`pip freeze` writes out every installed package pinned to its exact version. That text file is how your setup moves to another machine: a teammate runs `pip install -r requirements.txt` and gets precisely what you have, and at the checkpoint your Dockerfile replays the same file to build the container. Reproducibility from one small file — never from a description of your laptop.

Two habits follow. Commit `requirements.txt`; never commit `.venv/` (it is thousands of platform-specific binary files, and it will not even work on a different operating system). And re-run `pip freeze` every time you add a dependency, or the file quietly drifts out of date.

PART 3 · SEGMENT 02

Typed Python

What a type hint is — and is not

A type hint is an annotation that states what kind of value something is meant to hold: `name: str`, `monthly_income: float`, `-> float` for a return value. Python itself does not enforce them at runtime — it will not stop you passing a string where a float was declared. What they do is let *tools* help you: your editor autocompletes fields and flags mistakes as you type, and a checker like mypy can scan the whole file without running it.

In this session they pay off a second time. FastAPI reads exactly these annotations to validate incoming requests, serialise responses and generate your documentation. The same three words of typing do two jobs.

NO STUPID QUESTION

If Python ignores them at runtime, do type hints slow my program down?

Effectively no. They are recorded as metadata when the function is defined and then not consulted on every call. There is no per-call cost to annotating your code.

Enum — a fixed set of allowed values

An **enumeration** (Enum) is a class that lists a small, fixed set of named constants. Ours says a loan purpose may be working capital, equipment or a vehicle — and nothing else.

```
from enum import Enum

class LoanPurpose(str, Enum):
    WORKING_CAPITAL = "working_capital"
    EQUIPMENT = "equipment"
    VEHICLE = "vehicle"
```

Why bother, when a plain string would "work"? Because plain strings let every typo through. Write `"equipmnet"` and nothing complains until a customer notices. With an Enum you write `LoanPurpose.EQUIPMENT`: your editor autocompletes it, a typo is caught immediately, the list

of valid values exists in exactly one place, and anyone reading the class knows the full set of possibilities without hunting through the codebase.

The `(str, Enum)` part means each member is also a real string. That matters practically: it serialises straight to JSON as `"equipment"`, and it compares equal to the plain string `"equipment"`, so it behaves well at the edges of your system where data arrives as text.

NO STUPID QUESTION

Why are the Enum member names in capitals?

Convention, not requirement. Python writes constants in upper snake case, and Enum members are constants. The lowercase string on the right is the value that travels over the wire; the capitalised name on the left is how you refer to it in code.

dataclass — a class without the boilerplate

Writing a small class to hold data normally means writing an `__init__` that assigns every field, plus a readable `__repr__`, plus an equality method. The `@dataclass` decorator generates all of that from your annotated fields.

```
@dataclass
class Applicant:
    name: str
    monthly_income: float
    purpose: LoanPurpose
```

Three lines, and you can now write `Applicant("Asha", 45000, LoanPurpose.EQUIPMENT)`, print it usefully, and compare two instances by value. Note the crucial limit: a dataclass **annotates but does not validate**. Pass a string for `monthly_income` and it is stored exactly as given. That is fine for data you already trust — and not enough for data arriving from the internet, which is why Pydantic exists.

NO STUPID QUESTION

What is a decorator — that `@` thing?

A decorator wraps the thing defined beneath it and returns an enhanced version. `@dataclass` takes your bare class and hands back the same class with the generated methods added. `@app.get("/health")` takes your function and registers it with FastAPI as the handler for that URL. You will use decorators constantly today and write none of your own — knowing they mean "wrap and augment what follows" is enough.

Generics, and optional values

A bare `list` tells you almost nothing. `list[Applicant]` says what is *inside* the list, and that is what unlocks real help: your editor now knows each element has a `.name`, autocompletes it, and flags `.naem` as a mistake. Same for `dict[str, float]` — keys are strings, values are numbers. Always parameterise your containers.

Use `float | None` only where absence is genuinely meaningful, because every caller then has to handle the `None` case. Reaching for optional types everywhere spreads defensive checks through your whole codebase; requiring the value keeps the burden in one place.

PART 4 · SEGMENT 03

FastAPI: routes

The smallest possible service

A route is a plain Python function with a decorator naming the HTTP method and the URL path. That is genuinely all there is to it.

```
from fastapi import FastAPI

app = FastAPI(title="Loan Intake Service")

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

The `app` object is your application — a registry that the decorators add routes to. Returning a dictionary is enough; FastAPI converts it to JSON on the way out. A `/health` endpoint like this is a near-universal convention: something trivial that proves the service is alive, useful to load balancers, monitoring and — as you will see — Docker healthchecks.

Running it with uvicorn

```
uvicorn main:app --reload
```

Read the argument as "in the module `main`, use the object called `app`". `--reload` watches your files and restarts the server whenever you save, which is what makes live-coding bearable — never use it in production, where the restart-watching is pure overhead.

NO STUPID QUESTION

Why do I need uvicorn? Why not just `python main.py`?

Because your file defines an application but contains nothing that listens on the network. Something has to open a port, accept HTTP connections, parse requests and write responses back — that something is the server. **uvicorn** is that server; FastAPI is only the framework describing how to respond. The two speak a standard called **ASGI**, which is why any ASGI server can run any ASGI framework. (ASGI is the modern async successor to WSGI, the older synchronous standard behind Flask and Django.)

Documentation you did not write

Open `http://localhost:8000/docs` and a complete interactive API explorer is already there. Every route, parameter and data shape was derived from your type hints and Pydantic models. You can send real requests from that page — which is how a frontend developer explores your service without reading any of your source code.

Underneath it is **OpenAPI**, a standard machine-readable description of an HTTP API (served at `/openapi.json`). Because it is standardised, other tools can consume it — generating client code, test suites or documentation sites automatically.

NO STUPID QUESTION

What are `localhost` and `:8000`?

`localhost` is a name that always means "this same machine" (address `127.0.0.1`) — the request never leaves your computer. A **port** is a numbered door on that machine, so several programs can listen at once without collision. Our API listens on 8000, the web page on 3000, PostgreSQL on 5432. "Port already in use" simply means another program got to that door first.

PART 5 · SEGMENT 03

Pydantic: the contract

One class, three jobs

A Pydantic model is a class inheriting from `BaseModel`. From that single definition you get parsing of incoming JSON, validation of every field, and a description of the shape in your documentation.

```
class ApplicationIn(BaseModel):
    applicant_name: str = Field(min_length=2, max_length=100)
    monthly_income: float = Field(gt=0)
    amount_requested: float = Field(gt=0, le=10_00_000)
    purpose: str

class ApplicationOut(ApplicationIn):
    id: int
    status: str = "received"
```

`Field(...)` attaches the constraints: a name between 2 and 100 characters, an income strictly greater than zero (`gt`), a requested amount at most ten lakh (`le` — less than or equal). You wrote no validation code, no `if` statements, and no error messages, yet every one of those rules is now enforced and documented.

`ApplicationOut` inherits from `ApplicationIn`, so it has all four input fields plus the two the server assigns. Separating input from output is a deliberate discipline: clients must not be allowed to set the `id` or the `status`.

NO STUPID QUESTION

What is JSON, exactly?

A plain-text format for structured data — objects in braces, lists in brackets, strings in quotes — that essentially every language can read and write. It is how your web page and your Python service exchange data, because they cannot share memory or Python objects. Pydantic's job is turning that untrusted text into trustworthy Python objects, and back again.

Where each status code comes from

This distinction is worth internalising, because it tells you where to look when something breaks.

STATUS	RAISED BY	MEANING
201	Your decorator	Created. A new resource now exists — the right answer to a successful POST, not a bare 200.
422	Pydantic, before your code	The request shape is wrong — a missing field, a negative income, a value over the cap. Your function never ran.
404	Your own logic	The request was valid, but the thing asked for does not exist. You raise it with `HTTPException`.
400	You, generically	A general "bad request" when nothing more specific fits.

Read a 422 response body once, slowly: it names the field, the constraint that failed and a human message. That JSON is not noise — it is the contract the frontend codes against, and good frontends surface it to the user.

response_model guards the way out

Validation is usually discussed as an inbound concern, but `response_model=ApplicationOut` works on the outbound side: it validates and *filters* what you return. If your object carries an internal field that is not declared in the model, it is stripped before the client ever sees it. That is a genuine safety property — accidental data leaks are prevented structurally rather than by remembering to be careful.

How parameters are found

FastAPI works out where each argument comes from by looking at your signature alone. A name that also appears in the path — `/applications/{app_id}` with `app_id: int` — is a **path parameter**, and the text from the URL is converted to an integer for you. An argument typed with a Pydantic model is the **request body**. Anything else with a simple type and a default is read from the **query string**, as in `?purpose=equipment`.

PART 6 · SEGMENT 04

Testing with pytest

Why now, rather than later

Your tests protect the promises other people depend on. Once a web page is built against your 201 and 422 responses, a careless change to your service breaks their page — and a test is the thing that turns that silent breakage into an obvious red run before anyone ships it. Today the goal is modest: build the habit with one green run.

```
from fastapi.testclient import TestClient
from main import app, DB

client = TestClient(app)

def setup_function():
    DB.clear()                    # isolate every test

def test_negative_income_rejected():
    bad = {"applicant_name": "X", "monthly_income": -1,
           "amount_requested": 1000, "purpose": "vehicle"}
    assert client.post("/applications", json=bad).status_code == 422
```

Why it is so fast

`TestClient` calls your application *in process*. No server starts, no port opens, nothing touches the network — it hands requests directly to your ASGI app object and gives you the response. That is why two tests finish in under a second, and speed is what turns running tests into a reflex rather than a chore.

`setup_function` runs automatically before each test and clears the shared in-memory list, so tests cannot leak state into one another. Beyond that, pytest's whole API today is: name files `test_*.py`, name functions `test_*`, and use plain `assert`. It finds them itself.

NO STUPID QUESTION

My tests pass one at a time but fail together. What is happening?

Almost always shared state: one test leaves data behind and the next one counts it. That is exactly what `setup_function` (or a pytest *fixture*, its more powerful cousin) exists to prevent. A suite whose result depends on running order cannot be trusted, so fix it the moment you see it.

NO STUPID QUESTION

Why test that something *fails*? Isn't that backwards?

It is the more valuable test. A happy-path test only proves the good case still works; it will stay green even if you accidentally delete every validation rule you wrote. Asserting that a negative income returns 422 pins down the actual promise: bad data never becomes a stored application.

PART 7 · SEGMENT 05

The Integration Checkpoint

Nothing in this part is a new concept. It is assembly: the service you already wrote, a real database behind it, a web page in front of it, and Docker Compose holding the three together. "Done" means one command works from a clean clone — `docker compose up --build` — and the whole thing is pushed to Git.

PostgreSQL and the schema

Until now your applications lived in a Python list, which vanishes the moment the process stops. **PostgreSQL** is a relational database: data lives in tables of typed columns, survives restarts, and can be queried with SQL. Our `init.sql` creates the table and inserts two seed rows.

```
CREATE TABLE IF NOT EXISTS applications (
    id SERIAL PRIMARY KEY,
    applicant_name TEXT NOT NULL,
    monthly_income NUMERIC NOT NULL CHECK (monthly_income > 0),
    ...
    status TEXT NOT NULL DEFAULT 'received'
);
```

Read the guarantees in that table. `SERIAL PRIMARY KEY` means the database assigns each row a unique, automatically incrementing `id` — you never invent one. `NOT NULL` means the column must have a value. And `CHECK (monthly_income > 0)` is the same rule as Pydantic's `gt=0`, enforced one layer deeper.

That duplication is intentional, and it has a name: **defence in depth**. Pydantic gives a precise, friendly error to the person filling in the form. The database constraint is the last line, and it holds even if data arrives from somewhere that never passed through your API at all.

NO STUPID QUESTION

If I stop the containers, do I lose my data?

In our teaching setup, when the database container is removed, yes — its stored rows go with it, and `init.sql` re-runs from scratch on the next fresh start. That is convenient for a classroom, where you *want* a clean slate. For anything real you attach a named **volume** to the container: a storage area with its own lifetime, so data outlives the container. Be careful with `docker compose down -v` — the `-v` deletes volumes.

Note also that `init.sql` only runs on *first* initialisation of an empty database — it is not a migration system. Editing it later will not change an existing database.

Talking to the database safely

```
with get_conn() as conn:
    row = conn.execute(
        """INSERT INTO applications
             (applicant_name, monthly_income, amount_requested, purpose)
           VALUES (%(applicant_name)s, %(monthly_income)s,
                   %(amount_requested)s, %(purpose)s)
           RETURNING *""",
        payload.model_dump(),
    ).fetchone()
    conn.commit()
return ApplicationOut(**row)
```

Several things are worth naming here. `psycopg` is the driver — the library that lets Python speak PostgreSQL's protocol. `row_factory=dict_row` makes each result a dictionary rather than a bare tuple, which is why `ApplicationOut(**row)` works: the `**` unpacks the dictionary into keyword arguments. `RETURNING *` asks PostgreSQL to hand back the row it just inserted, including the `id` it generated. `with` guarantees the connection is closed even if an error is raised, and `commit()` makes the change permanent.

NO STUPID QUESTION

Why the strange `%(name)s` placeholders instead of an f-string?

Because building SQL by pasting values into a string is how **SQL injection** happens. If a value came from a user and contained SQL syntax, that syntax would become part of your query — able to read or destroy data. Placeholders keep the query and the data on separate tracks: the driver sends the SQL shape and the values independently, so a value can never be interpreted as a command. Never format user data into a query string, in any language, ever.

NO STUPID QUESTION

Aren't we supposed to use an ORM?

An ORM (SQLAlchemy, Django's ORM) maps database rows to Python objects so you write Python instead of SQL. They are excellent and you will meet them. We use raw SQL today deliberately: with two queries, an ORM would add a large new concept without teaching you anything about the actual conversation between your service and the database. Learn the conversation first; the abstraction makes more sense afterwards.

CORS — why the browser blocks your own API

Your page is served from `localhost:3000` and your API lives at `localhost:8000`. Because the *port* differs, browsers treat those as two different **origins** — and by default a page may not read responses from a different origin. This is a deliberate, long-standing browser protection: without it, any site you visited could quietly make authenticated requests to your bank in the background.

The fix is for the API to declare which origins are permitted, which is what `CORSMiddleware` does with `allow_origins=["http://localhost:3000"]`. Note who enforces this: the **browser** does. Your API is fine, your page is fine, and the same request from `curl` works perfectly — which is exactly why a CORS error feels so baffling the first time. It is not a bug in FastAPI; it is the browser waiting to be told the call is allowed.

The one that catches everyone: `db` versus `localhost`

Inside the Compose file, two different hostnames appear for two different journeys, and mixing them up is the single most common checkpoint failure.

```
api:
  environment:
    DATABASE_URL: postgresql://intake:intake@db:5432/intake   # inside
web:
  environment:
    NEXT_PUBLIC_API_URL: http://localhost:8000                # outside
```

Compose puts all three containers on one private network and gives each a DNS name equal to its service name. So from *inside* that network, the api reaches the database at the hostname `db`. Your browser, however, is not on that network — it runs on your machine, outside — so it uses the port that Compose *published* to your machine: `localhost:8000`.

If the api tried `localhost:5432`, "localhost" would mean the api's own container, where no database is running, and you would get a connection-refused error. Whenever you see that error at the checkpoint, check this first.

NO STUPID QUESTION

Why `NEXT_PUBLIC_` on that variable name?

Next.js only exposes environment variables to browser code when their names start with `NEXT_PUBLIC_`. Everything else stays server-side. It is a safety rail: secrets do not leak into the JavaScript bundle by accident. The corollary — never put a real secret in a `NEXT_PUBLIC_` variable, because it is public by definition.

NO STUPID QUESTION

Why does the Dockerfile say `--host 0.0.0.0`?

By default uvicorn listens only on `127.0.0.1` — connections originating from inside that same container. Nothing outside could reach it, including your browser. `0.0.0.0` means "listen on all network interfaces", which is what makes the published port actually work. It is a required line, not a decorative one.

"Started" is not "ready"

PostgreSQL takes a moment to initialise after its container starts. `depends_on` alone only controls start *order*, so the api can easily win a race against a database that is not yet accepting connections — and crash on its first query. The healthcheck runs `pg_isready` repeatedly until the database genuinely answers, and `condition: service_healthy` holds the api back until that passes. The race disappears.

This distinction between "the process has launched" and "the service can do its job" appears everywhere in infrastructure work. It is worth carrying with you.

Dockerfiles, and why the copy order matters

A Dockerfile is the recipe for an image, executed top to bottom, with each instruction producing a cached layer. Notice that we copy the dependency manifest and install *before* copying the source code:

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Docker reuses a cached layer whenever that step's inputs have not changed. Because the install step depends only on `requirements.txt`, editing your Python code does not invalidate it — the rebuild skips straight past installing and takes seconds. Reverse the two and every one-character edit reinstalls every dependency. This ordering trick is standard practice in every language's Dockerfiles.

Git: the last ten minutes

Three commands, three distinct meanings, often blurred together. `git add` stages the changes you intend to record. `git commit` writes that snapshot into your *local* history with a message. `git push` sends your local commits to the shared server. Work that is committed but never pushed still exists only on your laptop.

Your `.gitignore` lists what must never be committed — `.venv/`, `node_modules/`, `__pycache__/`. All three are large, machine-specific and fully regenerable from the manifests you *do* commit. The principle: version the recipe, never the build output.

REFERENCE

Glossary

Every term the session assumes, in one place.

TERM	IN ONE SENTENCE
API	A defined way for one program to request something from another over a network.
REST	Conventions for HTTP APIs: URLs name resources, verbs say what you are doing to them.

TERM	IN ONE SENTENCE
ASGI	The standard interface between an async Python web framework and its server.
uvicorn	The ASGI server that actually listens on a port and runs your FastAPI app.
OpenAPI	A machine-readable description of an HTTP API; the source of the `/docs` page.
Pydantic	The library that parses, validates and documents data at your service's boundary.
Type hint	An annotation stating the intended type; used by tools, not enforced by Python.
Enum	A class defining a fixed, named set of allowed values.
dataclass	A decorator that generates a data-holding class from annotated fields — no validation.
Decorator	An `@` annotation that wraps the function or class below it to add behaviour.
venv	A per-project folder holding its own Python and packages; survives restarts, needs re-activating.
pip	Python's package installer.
pytest	The test runner that discovers `test_*` functions and reports pass or fail.
Fixture	Reusable pytest setup/teardown that supplies a test with what it needs.
Image	The frozen recipe for a container, stored on disk.
Container	A running, isolated instance of an image, sharing your machine's kernel.
Docker Compose	Runs several containers as one system from a YAML file; `docker compose`, with a space.
Volume	Container storage with its own lifetime, so data outlives the container.
Healthcheck	A command Docker repeats to decide whether a service is truly ready, not merely started.

TERM	IN ONE SENTENCE
Origin	Scheme + host + port; differing in any one makes it a different origin to a browser.
CORS	The mechanism by which an API tells browsers which other origins may read its responses.
SQL injection	An attack enabled by pasting user data into SQL text; prevented by parameterised queries.
Migration	A versioned, repeatable change to a database schema — what production uses instead of `init.sql`.
ORM	A library mapping database rows to objects so you write code instead of SQL.

IF YOU REMEMBER NOTHING ELSE

Six things worth keeping

1. A virtual environment is a folder — it survives a restart; only the *activation* needs repeating.
2. Type hints are ignored by Python at runtime, and read by your editor, mypy and FastAPI.
3. Validate at the boundary with Pydantic, and again in the database with constraints.
4. 422 comes from Pydantic before your handler; 404 comes from your own logic.
5. Inside a Compose network, address services by name (`db`); from the browser, use the published port on `localhost`.
6. Commit the recipe (`requirements.txt`, Dockerfiles), never the build output (`.venv/`, `node_modules/`).

Anything still unclear is worth asking about — in the channel, in the next session, or in your one-minute retro. A question you carry quietly costs far more than a question asked out loud.